Pregrado en Ingeniería de Sistemas e Informática
Universidad Pontificia Bolivariana Seccional Bucaramanga

Simulador de crédito educativo para la Universidad Pontificia Bolivariana Seccional Bucaramanga

Flórez Durán, D.E., Carvajal Barragán, D.R., Ordóñez Ferreira, J.D., Mora Patiño, J.D., Pérez Gómez, S., Rodríguez Castro, G.M., Sastoque Cadena, G.A., Arenas Avendaño, J.P., Cifuentes Cárdenas, S.

Universidad Pontificia Bolivariana Seccional Bucaramanga, km 7 autopista a Piedecuesta, Floridablanca – Colombia

Resumen— El acceso a la educación superior no depende únicamente de la oferta académica, sino también de la claridad que tienen los aspirantes y estudiantes sobre las alternativas de financiación disponibles. En la Universidad Pontificia Bolivariana Seccional Bucaramanga se identificó que muchas consultas relacionadas con matrícula, cuotas, intereses, descuentos, becas y proyecciones de pago eran atendidas de forma manual por el área financiera, lo que generaba tiempos de respuesta prolongados, dependencia de herramientas estáticas y dificultad para actualizar las condiciones financieras cuando cambiaban las políticas institucionales. Frente a esta situación, se desarrolló una plataforma web institucional para la simulación de créditos educativos, orientada a brindar autonomía al usuario y a reducir la carga operativa del área administrativa.

El proyecto se abordó como una investigación aplicada de alcance descriptivo, mediante un modelo incremental con prototipado evolutivo y apoyo en artefactos seleccionados de Scrum, tales como Product Backlog, Sprint Backlog, historias de usuario, Daily Scrum, tableros de seguimiento y métricas de avance. El desarrollo se organizó en tres sprints, en los cuales se trabajaron la arquitectura base, la autenticación, el motor de cálculo financiero, la parametrización administrativa, el historial de simulaciones, las exportaciones, las pruebas y el despliegue institucional. La plataforma fue construida con React, Vite, Tailwind CSS, TypeScript, Node.js, Express, MySQL, Sequelize, JWT, BCrypt, LDAP, GitHub Actions, Nginx y PM2.

Como resultado, se implementó un sistema funcional con acceso para invitado, estudiante y administrador, capaz de calcular escenarios de financiación, generar tablas de amortización, aplicar reglas de cuota inicial, manejar modalidades con y sin intereses, guardar simulaciones y permitir la administración de parámetros financieros. El proyecto alcanzó 82 historias completadas de 84 planeadas y 692 puntos terminados de 703 comprometidos, lo que equivale aproximadamente al 98 % del alcance funcional. Finalmente, las pruebas unitarias, funcionales, de integración y de regresión permitieron validar la precisión de los cálculos y la estabilidad general del sistema antes de su entrega y despliegue en producción.

Palabras clave— amortización francesa, crédito educativo, plataforma web, simulación financiera, transformación digital.

Código JEL: C63, I22, O33.

Abstract— Access to higher education depends not only on academic programs, but also on the financial clarity that applicants and students have when making enrollment decisions. At Universidad Pontificia Bolivariana Bucaramanga, the financial department frequently handled manual inquiries related to tuition fees, installments, interest rates, scholarships, discounts and payment projections. This situation produced long response times, dependence on static tools and difficulties when institutional financial policies changed. To address this problem, an institutional web platform for educational credit simulation was developed, aiming to provide user autonomy and reduce the operational workload of the administrative area.

The project was carried out as an applied research project with a descriptive scope, using an incremental model with evolutionary prototyping and selected Scrum artifacts, such as Product Backlog, Sprint Backlog, user stories, Daily Scrum meetings, tracking boards and progress metrics. The development process was organized into three sprints, covering the base architecture, authentication, financial calculation engine, administrative parameter management, simulation history, exports, testing and institutional deployment. The platform was built using React, Vite, Tailwind CSS, TypeScript, Node.js, Express, MySQL, Sequelize, JWT, BCrypt, LDAP, GitHub Actions, Nginx and PM2.

As a result, a functional system was implemented with guest, student and administrator access. The platform can calculate financing scenarios, generate amortization tables, apply initial payment rules, manage interest and no-interest modalities, store simulations and allow administrators to update financial parameters. The project completed 82 out of 84 planned user stories and 692 out of 703 committed points, reaching approximately 98% of the expected functional scope. Finally, unit, functional, integration and regression tests helped validate the accuracy of the calculations and the overall stability of the system before its final delivery and production deployment.

Keywords— educational credit, financial simulation, French amortization, digital transformation, web platform.

1. Introducción

En los procesos de admisión universitaria, la información financiera tiene un papel importante porque influye directamente en la decisión de matrícula de los aspirantes. Aunque un estudiante pueda estar interesado en un programa académico, la falta de claridad sobre costos, cuotas, intereses, descuentos, becas y formas de pago puede convertirse en una barrera de entrada. Esta situación no solo afecta al usuario, sino también a las áreas administrativas de las instituciones de educación superior, que deben responder de manera repetitiva consultas relacionadas con escenarios de financiación.

En la Universidad Pontificia Bolivariana Seccional Bucaramanga se identificó esta problemática dentro del área financiera. Los aspirantes y estudiantes necesitaban conocer de forma rápida cuánto debían pagar como cuota inicial, cuál sería el valor financiado, cómo se distribuirían las cuotas y qué diferencia existía entre una modalidad con intereses y una modalidad sin intereses. Sin embargo, parte de esta orientación dependía de procesos manuales, herramientas estáticas o archivos que requerían intervención del personal encargado. Esto generaba una carga operativa considerable y hacía más lenta la atención de dudas frecuentes.

El problema también se relacionaba con la actualización de reglas financieras. Cuando cambiaban las tasas, los porcentajes mínimos de cuota inicial o las condiciones para determinados niveles de formación, era necesario ajustar manualmente la información utilizada para orientar a los usuarios. En ese sentido, la necesidad no era únicamente crear una calculadora de cuotas, sino construir una plataforma institucional que permitiera centralizar reglas, parametrizar condiciones, generar resultados comprensibles y mantener trazabilidad sobre las simulaciones realizadas.

A partir de esta necesidad surgió el proyecto del Simulador de Crédito Educativo UPB, desarrollado como parte de Proyecto Integrador I. La solución se planteó como una plataforma web que permitiera a los usuarios simular escenarios de crédito académico de forma autónoma y al área financiera administrar los parámetros del sistema sin depender de modificaciones técnicas constantes. El sistema debía ofrecer una experiencia clara, rápida y confiable para tres tipos de usuario: invitado, estudiante autenticado y administrador.

La pregunta que orientó el desarrollo fue: ¿cómo implementar una solución tecnológica que permita la simulación de créditos académicos y la generación de resúmenes financieros, con el propósito de brindar autonomía a los estudiantes y reducir la carga operativa del área financiera de la Universidad Pontificia Bolivariana Seccional Bucaramanga? Esta pregunta fue tomada como eje para el levantamiento de requerimientos, el diseño de la arquitectura, la implementación del motor de cálculo y la validación del sistema.

El objetivo general del proyecto fue implementar una plataforma web institucional para la simulación de créditos académicos y la generación de resúmenes financieros en la Universidad Pontificia Bolivariana Seccional Bucaramanga, integrando los lineamientos financieros vigentes para facilitar la toma de decisiones de los estudiantes y disminuir la carga administrativa del área financiera. Este objetivo se desarrolló mediante cuatro objetivos específicos: identificar requerimientos funcionales y no funcionales, diseñar la arquitectura y los prototipos de interfaz, desarrollar el motor de cálculo y los módulos funcionales, y validar la plataforma mediante pruebas antes de su despliegue.

El presente artículo describe el proceso completo de análisis, diseño, desarrollo, validación y despliegue de la plataforma. Para ello se presenta una aproximación teórica relacionada con crédito educativo, simulación financiera, amortización, arquitectura web, seguridad, bases de datos, metodologías ágiles y marco legal. Posteriormente se explica la metodología aplicada, los resultados obtenidos por sprint, las pruebas realizadas, el despliegue institucional y la discusión de los principales hallazgos, limitaciones y aprendizajes del proyecto.

La Figura 1 resume las etapas metodológicas aplicadas durante el desarrollo del proyecto.

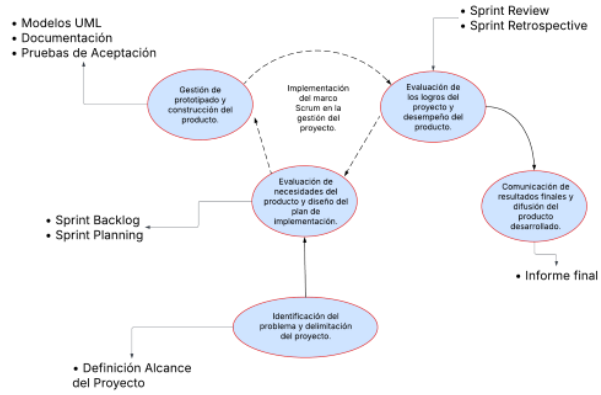


Figura 1. Etapas metodológicas para el desarrollo del proyecto.
Fuente: Elaboración propia, 2026.

2. Aproximación teórica

2.1. Crédito educativo y orientación financiera

El crédito educativo es una alternativa de financiación que permite cubrir total o parcialmente el valor de una matrícula mediante pagos distribuidos en el tiempo. En el contexto universitario, este tipo de financiación permite que los aspirantes puedan acceder a programas académicos aun cuando no cuentan con la totalidad del valor de la matrícula al momento de ingresar. Sin embargo, para que esta alternativa sea realmente útil, el estudiante debe comprender las condiciones del crédito, el valor de la cuota inicial, el monto financiado, los intereses y el calendario de pagos (ICETEX s. f.).

La orientación financiera en instituciones de educación superior no debe limitarse únicamente a informar un valor total de matrícula. También debe ayudar al usuario a comparar escenarios, identificar el impacto de una beca o descuento, entender la diferencia entre financiar con intereses o sin intereses, y visualizar cómo se distribuye el pago durante el periodo académico. Por esta razón, una herramienta de simulación financiera puede aportar valor tanto al usuario como a la institución.

Desde la perspectiva del usuario, el simulador permite tomar decisiones con mayor claridad. Desde la perspectiva administrativa, reduce la cantidad de consultas repetitivas y permite que el área financiera enfoque su tiempo en casos que requieren atención personalizada. En este proyecto, el crédito educativo se asumió como un proceso de orientación, no como una aprobación automática de crédito. Por lo tanto, los resultados generados por la plataforma tienen carácter informativo y están sujetos a validación institucional.

2.2 Simulación financiera y amortización francesa

La simulación financiera consiste en representar escenarios de pago a partir de variables como valor de matrícula, descuentos, becas, cuota inicial, tasa de interés, número de cuotas y modalidad de financiación. En el caso del simulador desarrollado, el objetivo principal fue calcular el valor neto a pagar, el valor financiado, las cuotas periódicas, los intereses y el saldo restante después de cada pago.

Uno de los conceptos utilizados fue la amortización francesa, un modelo financiero caracterizado por cuotas periódicas constantes cuando existe una tasa de interés aplicada al saldo financiado. En este sistema, al inicio del crédito una parte mayor de la cuota corresponde a intereses y una parte menor a capital; con el paso del tiempo, la proporción se invierte hasta cancelar el saldo. Este modelo facilita la generación de tablas de amortización porque permite mostrar en cada fila el número de cuota, la fecha estimada, el capital abonado, el interés causado, el valor total de la cuota y el saldo pendiente (Banco de la República, s. f.).

En el proyecto, la tabla de amortización no se diseñó únicamente como un resultado técnico, sino como un elemento de interpretación para el usuario. Por eso se incluyeron resúmenes, valores destacados y gráficos de apoyo que permiten comprender rápidamente cuánto se paga al inicio, cuánto se financia, cuánto corresponde a intereses y cuál es el total estimado del plan de pago (Gitman & Zutter, 2012).

2.3. Arquitectura web y separación de responsabilidades

Las aplicaciones web modernas suelen organizarse bajo una arquitectura cliente-servidor. En este modelo, el frontend se encarga de la interfaz gráfica y la experiencia de usuario, mientras que el backend procesa la lógica de negocio, valida los datos, administra la seguridad y se comunica con la base de datos. Esta separación permite que cada parte del sistema evolucione de manera más ordenada (Fowler, 2002).

En el simulador se utilizó React con Vite, Tailwind CSS y TypeScript para el frontend. Esta combinación permitió construir una interfaz modular, responsiva y alineada con la identidad visual institucional. En el backend se utilizó Node.js con Express y TypeScript, siguiendo principios de arquitectura limpia para separar dominio, casos de uso, infraestructura y rutas de API. Esta decisión facilitó que el motor de cálculo financiero no quedara mezclado con la interfaz ni con detalles de persistencia.

La comunicación entre frontend y backend se realizó mediante servicios tipo REST, utilizando JSON como formato de intercambio de datos. Este enfoque permitió exponer endpoints para simular, guardar resultados, consultar historial, administrar parámetros y validar usuarios según su rol. Además, la documentación de API con Swagger facilitó la revisión técnica y las pruebas durante el desarrollo.

2.4. Gestión de datos, trazabilidad y persistencia

La base de datos cumplió un papel fundamental porque el sistema no solo debía calcular resultados en pantalla, sino también gestionar usuarios, roles, simulaciones, parámetros financieros y registros administrativos. Para ello se utilizó MySQL como base relacional y Sequelize como ORM. El uso de un ORM permitió mapear entidades del sistema a tablas relacionales, mantener una estructura

más organizada y reducir la escritura repetitiva de consultas SQL.

También se consideraron mecanismos de trazabilidad, auditoría y eliminación lógica. La trazabilidad fue importante porque el área financiera necesitaba claridad sobre qué parámetros estaban activos, qué simulaciones se habían guardado y qué acciones administrativas se habían realizado. La eliminación lógica o soft delete permitió marcar registros como eliminados sin borrarlos físicamente de inmediato, lo cual es útil cuando se requiere conservar evidencia o evitar pérdida accidental de información.

La gestión de datos se complementó con políticas de retención. En los requerimientos se planteó que no todas las simulaciones debían almacenarse indefinidamente, especialmente aquellas no formalizadas o generadas como pruebas. Esta decisión buscó evitar sobrecarga innecesaria de almacenamiento y mantener la base de datos con información relevante.

2.5. Seguridad, autenticación y autorización

La seguridad fue un componente relevante porque el sistema administra datos personales, información financiera y funciones administrativas. Para el acceso de usuarios se implementaron mecanismos de autenticación y autorización. La autenticación permite verificar la identidad del usuario, mientras que la autorización define qué acciones puede realizar según su rol.

En el sistema se utilizaron JSON Web Tokens para la gestión de sesión, BCrypt para el cifrado de contraseñas y control de acceso basado en roles. Los roles principales fueron invitado, estudiante y administrador. El invitado puede realizar simulaciones básicas sin guardar historial ni acceder a funciones avanzadas. El estudiante autenticado puede guardar y consultar simulaciones. El administrador puede gestionar parámetros financieros, revisar información y acceder a módulos restringidos (OWASP Foundation, s. f.).

Adicionalmente, para el acceso administrativo se integró LDAP como mecanismo de autenticación institucional. Esta integración permitió alinear el ingreso de administradores con el directorio de la universidad, separando el proceso de autenticación de la autorización interna del sistema. En otras palabras, LDAP valida la identidad institucional y la base de datos local controla el rol y los permisos dentro de la aplicación.

2.6 Modelo incremental, prototipado evolutivo y artefactos Scrum

El proyecto no se desarrolló como un proceso lineal, sino como un trabajo incremental. Esto permitió construir primero una versión base del sistema y luego ampliarla con nuevas funcionalidades de acuerdo con la retroalimentación del cliente. El prototipado evolutivo fue importante porque las reglas financieras y las necesidades de interfaz fueron ajustándose durante el proyecto.

También se utilizaron artefactos de Scrum, como Product Backlog, Sprint Backlog, historias de usuario, reuniones de seguimiento, tableros en GitHub Projects, métricas de velocidad, burnup y burndown. Estos elementos permitieron organizar el trabajo del equipo, asignar responsabilidades, estimar esfuerzo y medir el avance por sprint (Schwaber & Sutherland, 2020).

En este artículo se habla de “modelo incremental con prototipado evolutivo apoyado en artefactos Scrum”, porque el equipo no aplicó Scrum de manera completa y estricta en todos sus eventos formales, pero sí utilizó varios de sus instrumentos para planear, ejecutar y hacer seguimiento al desarrollo.

2.7 Marco legal y naturaleza del simulador

El simulador no constituye una aprobación automática de crédito ni genera una obligación financiera por sí mismo. Su función es orientar al usuario mediante escenarios estimados de financiación. Esta aclaración es importante porque los resultados dependen de las políticas vigentes del área financiera y de la validación institucional posterior.

En cuanto al tratamiento de datos personales, el proyecto se relaciona con la Ley 1581 de 2012, que regula la protección de datos personales en Colombia. Además, al tratar información financiera, también se tuvo en cuenta la Ley 1266 de 2008, relacionada con el manejo de información financiera, crediticia y comercial (Congreso de Colombia, 2008). Por esta razón, el sistema debía manejar términos y condiciones, control de acceso, protección de credenciales y separación de permisos según rol.

3. Metodología

3.1. Enfoque y alcance

El proyecto se desarrolló bajo un enfoque de investigación aplicada, ya que buscó resolver una necesidad concreta del área financiera de la Universidad Pontificia Bolivariana Seccional Bucaramanga mediante el diseño, implementación y validación de una solución tecnológica. No se trató únicamente de estudiar el problema, sino de construir un producto funcional que pudiera ser usado por aspirantes, estudiantes y administradores.

El alcance fue descriptivo y tecnológico. Descriptivo, porque se analizaron los procesos, necesidades, reglas de negocio, usuarios y resultados generados por el sistema.

Tecnológico, porque se diseñó e implementó una plataforma web con módulos funcionales, base de datos, servicios de backend, interfaz de usuario, pruebas y despliegue institucional.

La metodología combinó prototipado evolutivo, desarrollo incremental y artefactos seleccionados de Scrum. Esta combinación permitió que el equipo avanzara por entregas, recibiera retroalimentación del cliente y ajustara el producto de acuerdo con los cambios solicitados durante las reuniones.

3.2 Población, muestra y usuarios del sistema

La población objeto del proyecto estuvo conformada por los usuarios relacionados con el proceso de financiación educativa de la Universidad Pontificia Bolivariana Seccional Bucaramanga. Dentro de esta población se identificaron tres perfiles principales:

Invitado: aspirante o usuario externo que desea realizar una simulación rápida sin crear cuenta ni iniciar sesión.

Estudiante: usuario autenticado que puede realizar simulaciones, guardar resultados y consultar su historial.

Administrador: usuario del área financiera o institucional encargado de gestionar parámetros, revisar simulaciones, administrar información y mantener actualizadas las reglas financieras.

La muestra de validación estuvo representada por escenarios de prueba definidos por el equipo y por las revisiones realizadas con el cliente durante las reuniones de seguimiento. Estas reuniones permitieron verificar si la plataforma respondía a las necesidades reales del proceso financiero.

3.3 Variables e indicadores de análisis

Para evaluar el proyecto se tuvieron en cuenta variables funcionales, técnicas y de gestión. Entre las variables funcionales se incluyeron precisión del cálculo, claridad de los resultados, correcta aplicación de reglas financieras, generación de tabla de amortización y disponibilidad de funciones según rol. Entre las variables técnicas se consideraron estabilidad del sistema, respuesta de la API, seguridad de acceso, persistencia de datos y despliegue en ambiente institucional. En cuanto a la gestión, se midieron historias de usuario completadas, puntos desarrollados, horas proyectadas, horas invertidas y cumplimiento por sprint.

Tabla 1. Variables e indicadores del proyecto

Variable	Indicador	Verificación
Cumplimiento funcional	HU completadas	Product Backlog y Sprint Backlog
Avance del proyecto	Puntos por sprint	GitHub Projects
Precisión financiera	Cuotas e intereses	Casos de prueba
Seguridad	Roles y autenticación	Pruebas funcionales
Usabilidad	Flujo de simulación	Validación cliente
Estabilidad	Integración frontend/backend	Pruebas de integración
Despliegue	Acceso productivo	URL institucional

Fuente: Elaboración propia con base en artefactos del proyecto, 2026.

3.4 Técnicas de recolección de información

La información del proyecto se recolectó mediante reuniones con el cliente, actas firmadas, análisis de documentos institucionales, historias de usuario, revisión de prototipos, pruebas del sistema y seguimiento en GitHub Projects. Las actas de reunión fueron especialmente importantes porque permitieron documentar acuerdos, cambios solicitados, validaciones parciales y aprobación progresiva del sistema.

También se utilizaron documentos técnicos como el Project Charter, el documento de especificación de requisitos, el Product Backlog, los Sprint Backlogs, el plan de pruebas, los diagramas UML, el modelo entidad-relación, el modelo relacional y la documentación de despliegue. Estos documentos permitieron mantener trazabilidad entre la necesidad inicial, los requerimientos, la implementación y los resultados finales.

3.5 Fases del desarrollo

El proyecto se organizó en tres sprints principales.

Durante el Sprint 1, realizado entre el 11 de febrero y el 3 de marzo, se consolidó la base técnica del sistema. En esta etapa se trabajó la arquitectura inicial, la autenticación, los roles, la base de datos, el formulario de simulación y la primera versión de la landing informativa del proyecto. Este sprint permitió tener una primera base funcional sobre la cual se construyeron los módulos posteriores.

Durante el Sprint 2, realizado entre el 5 y el 25 de marzo, se implementó el núcleo funcional del simulador. Se

desarrolló el motor de cálculo financiero, la parametrización administrativa, los resultados visuales, el acceso como invitado, el historial de simulaciones, las exportaciones y varios avances de infraestructura. Este sprint fue el más fuerte en términos de puntos completados, porque concentró la mayor parte de las funcionalidades centrales del producto.

Durante el Sprint 3, realizado entre el 6 y el 26 de abril, se priorizaron las pruebas, la integración institucional, el despliegue, la base de datos productiva, LDAP, los servicios de red, el monitoreo, los backups y la documentación final. En esta etapa se preparó el sistema para su entrega, se ajustaron elementos técnicos finales y se validó el funcionamiento integral de la plataforma.

3.6. Herramientas y tecnologías utilizadas

Tabla 2. Tecnologías utilizadas en el proyecto

Categoría	Tecnología	Propósito
Frontend	React + Vite	Construcción de la interfaz web
Estilos	Tailwind CSS	Diseño responsivo e identidad visual
Lenguaje	TypeScript	Tipado estático y mayor control del código
Backend	Node.js + Express	API REST y lógica de negocio
Base de datos	MySQL	Persistencia de usuarios, parámetros y simulaciones
ORM	Sequelize	Mapeo de entidades y gestión de modelos
Seguridad	JWT + BCrypt	Sesiones, autenticación y cifrado
Integración institucional	LDAP	Autenticación administrativa
Documentación API	Swagger	Visualización y prueba de endpoints
Pruebas	Jest + Postman	Validación de servicios y lógica
Control de versiones	Git + GitHub	Gestión del código fuente
Automatización	GitHub Actions	Validación, build y despliegue
Infraestructura	Nginx + PM2	Servidor web y gestión de procesos
Despliegue	Servidor institucional	Publicación del producto final

Fuente: Elaboración propia con base en el marco tecnológico del proyecto, 2026.

4. Resultados

4.1 Consolidación de requerimientos y alcance funcional

Uno de los primeros resultados del proyecto fue la consolidación del Product Backlog. El sistema se organizó en 22 épicas, 84 historias de usuario y 703 puntos de historia. Esta distribución permitió clasificar el trabajo por módulos y priorizar las funcionalidades de acuerdo con su importancia para el cliente.

Las historias de usuario cubrieron funciones relacionadas con acceso, autenticación, simulación, resultados financieros, historial, administración, exportaciones, seguridad, documentación, pruebas e infraestructura. Esta forma de organización ayudó a que el equipo no desarrollara funcionalidades aisladas, sino elementos conectados con necesidades reales del proceso financiero.

Tabla 3. Distribución general del Product Backlog

Elemento	Cantidad
Épicas	22
Historias de usuario	84
Puntos de historia	703
Sprints ejecutados	3
Roles principales del sistema	3

Fuente: Product Backlog consolidado del proyecto, Autores, 2026.

4.2 Resultados del Sprint 1

El Sprint 1 tuvo como objetivo consolidar la base técnica del simulador mediante la arquitectura inicial, autenticación, roles, formulario de simulación, base de datos y primera landing page. Durante este sprint se planearon 30 historias de usuario, se completaron las 30 y se alcanzaron 121 puntos de historia. El cumplimiento fue del 100 %, con 659 horas proyectadas y 606 horas invertidas.

Este resultado fue importante porque permitió construir la estructura mínima del sistema. Al finalizar el sprint, el equipo ya contaba con una base de frontend, backend, rutas iniciales, estructura de base de datos, autenticación y primeras pantallas funcionales. También se empezó a validar el flujo de simulación, aunque todavía no se encontraba completo en términos de parametrización avanzada y resultados finales.

La figura 2 muestra el avance acumulado del Sprint 1 mediante la gráfica BurnUp.

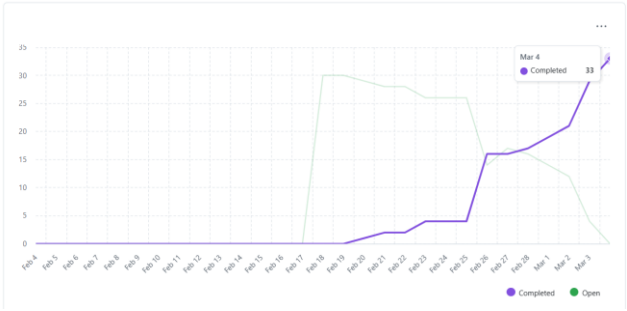


Figura 2. BurnUp del Sprint 1.
Fuente: GitHub Projects y Sprint Backlog I, Autores, 2026.

4.3 Resultados del Sprint 2

El Sprint 2 tuvo como objetivo implementar el núcleo funcional del simulador: parametrización financiera, motor de cálculo, resultados visuales, acceso invitado, historial, administración, exportaciones y avances de infraestructura. En este sprint se planearon 31 historias de usuario, se completaron las 31 y se alcanzaron 351 puntos de historia. El cumplimiento fue del 100 %, con 497 horas proyectadas y 441 horas invertidas.

Este fue el sprint con mayor volumen funcional. El motor de cálculo financiero permitió procesar datos de entrada como valor de matrícula, beca o descuento, cuota inicial, modalidad de pago y número de cuotas. A partir de estos datos, el sistema generó el valor neto, el valor financiado, el total a pagar, la tabla de amortización y los resultados visuales para el usuario.

También se fortaleció el módulo administrativo, especialmente la parametrización financiera. Esta funcionalidad fue clave porque el área financiera necesitaba actualizar tasas, reglas y condiciones sin depender de cambios directos en el código. De esta manera, el sistema pasó de ser una calculadora rígida a una plataforma configurable.

La figura 3 presenta el comportamiento de avance del Sprint 2.

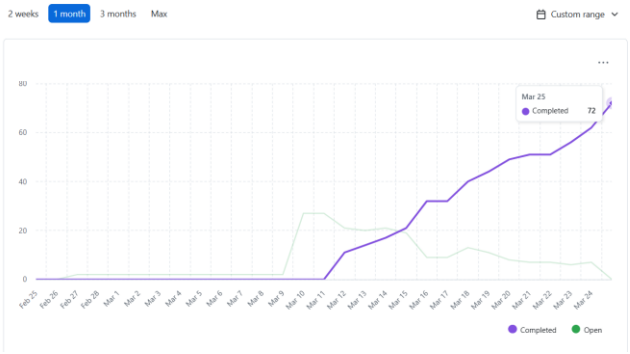


Figura 3. BurnUp del Sprint 2.
Fuente: GitHub Projects y Sprint Backlog II, Autores, 2026.

4.4 Resultados del Sprint 3

El Sprint 3 tuvo como objetivo validar, desplegar e integrar técnicamente el sistema mediante pruebas, base de datos institucional, LDAP, servicios de red, monitoreo, backups y documentación final. En esta etapa se planearon 23 historias de usuario, se completaron 21, se comprometieron 231 puntos y se completaron 220. El cumplimiento fue del 95 %, con 390 horas proyectadas y 426 horas invertidas.

Aunque dos historias quedaron aplazadas por dependencias externas de infraestructura y configuración institucional, el sprint permitió cerrar la mayor parte del alcance funcional. También se realizaron pruebas, documentación final, ajustes de despliegue e integración con servicios institucionales. El aumento de horas invertidas frente a las proyectadas se explica por la complejidad de integrar componentes externos y por los ajustes necesarios para preparar el sistema en un ambiente más cercano a producción.

La figura 4 resume el avance registrado durante el Sprint 3.

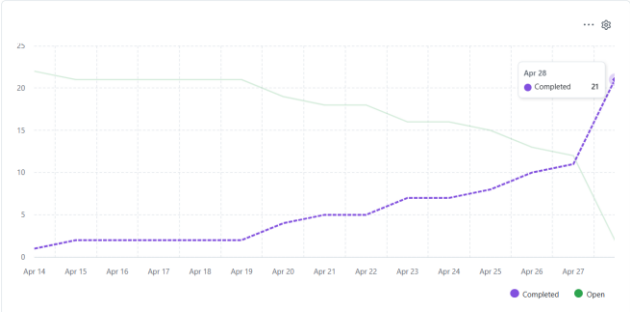


Figura 4. BurnUp del Sprint 3.
Fuente: GitHub Projects y Sprint Backlog III, Autores, 2026.

4.5 Cumplimiento global del proyecto

Al consolidar los tres sprints, el proyecto completó 82 de 84 historias de usuario y 692 de 703 puntos de historia. Esto representa aproximadamente un 97,6 % de cumplimiento por historias y un 98,4 % de cumplimiento por puntos. Por esta razón, puede afirmarse que el proyecto alcanzó cerca del 98 % del alcance planificado.

Tabla 4. Resumen de resultados por sprint

Sprint	HU plan.	HU compl.	Pts plan.	Pts compl.	Cumpl.	Hrs plan.	Hrs inv.
Sprint 1	30	30	121	121	100 %	659	606
Sprint 2	31	31	351	351	100 %	497	441
Sprint 3	23	21	231	220	95 %	390	426
Total	84	82	703	692	≈98 %	1546	1473

Fuente: Elaboración propia con base en Sprint Backlogs y métricas del equipo, 2026.

El resultado global muestra que el equipo logró mantener un ritmo de trabajo estable durante los dos primeros sprints y enfrentar una mayor complejidad en el sprint final. El Sprint 3 exigió más horas de las proyectadas porque incluyó pruebas, despliegue, integración institucional, LDAP, base de datos, servicios de red y documentación final.

4.6 Funcionalidades implementadas

El sistema final integró módulos para tres perfiles de usuario. El modo invitado permitió realizar una simulación básica sin crear cuenta. El modo estudiante permitió acceder a funciones autenticadas, guardar simulaciones y consultar historial. El modo administrador permitió gestionar parámetros financieros y revisar información relevante para la operación del sistema.

La figura 5 muestra la interfaz principal del simulador desplegado.

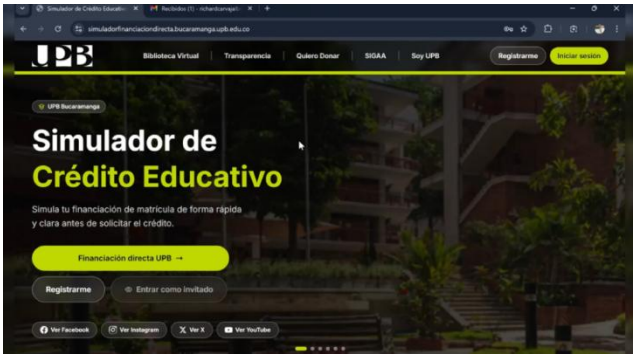


Figura 5. Interfaz principal del Simulador de Crédito Educativo.
Fuente: Captura del sistema desplegado, Autores, 2026.

Tabla 5. Módulos funcionales desarrollados

Módulo	Funcionalidad principal	Usuario asociado
Landing institucional	Presentación del sistema y acceso a opciones principales	Invitado, estudiante
Registro e inicio de sesión	Autenticación y control de acceso	Estudiante, administrador
Simulación como invitado	Cálculo básico sin guardar historial	Invitado
Simulación autenticada	Cálculo completo y opción de guardar resultados	Estudiante
Motor financiero	Cálculo de cuota inicial, valor financiado, intereses y tabla de amortización	Sistema
Historial de simulaciones	Consulta de simulaciones guardadas	Estudiante
Exportación	Generación de resultados en formatos descargables	Estudiante, administrador
Panel administrativo	Gestión de parámetros financieros y usuarios	Administrador

Módulo	Funcionalidad principal	Usuario asociado
Seguridad	JWT, BCrypt, roles y LDAP	Sistema
Documentación API	Endpoints documentados en Swagger	Equipo técnico

Fuente: Elaboración propia, 2026.

4.7 Reglas de negocio implementadas

El simulador incorporó reglas financieras definidas durante las reuniones con el cliente. Entre ellas se incluyeron porcentajes mínimos de cuota inicial, modalidades con y sin intereses, límites de cuotas, tratamiento de becas o descuentos, validaciones por tipo de estudiante y parametrización de tasas según nivel o programa.

Tabla 6. Principales reglas de negocio del simulador

Regla	Descripción
Cuota inicial mínima	Para programas generales se manejó una cuota inicial mínima del 20 %. Para UPB Tech se manejó una cuota inicial mínima del 15 %.
Modalidad sin intereses	Permite dividir el valor financiado en un número limitado de cuotas sin aplicar tasa de interés.
Modalidad con intereses	Aplica tasa de interés al valor financiado según parámetros definidos por el área financiera.
Beca o descuento	Permite descontar un valor o porcentaje antes de calcular el valor neto de matrícula.
Valor financiado	Corresponde al valor neto de matrícula menos la cuota inicial.
Tabla de amortización	Muestra cuota, capital, interés y saldo pendiente por periodo.
Parametrización administrativa	Permite actualizar tasas, vigencias y condiciones sin modificar el código fuente.
Acceso por rol	Las funciones disponibles cambian según el usuario sea invitado, estudiante o administrador.
Retención de datos	Las simulaciones no formalizadas o antiguas pueden gestionarse con políticas de eliminación y trazabilidad.

Fuente: Elaboración propia con base en requerimientos y actas del proyecto, 2026.

4.8 Motor de cálculo financiero

El motor de cálculo fue uno de los componentes principales del sistema. Su función fue recibir los datos ingresados por el usuario, validar las reglas financieras y generar una simulación comprensible. Para ello se calcularon valores como matrícula neta, cuota inicial, valor financiado, cantidad de cuotas, interés, valor total y saldo final.

En una simulación general, el sistema realiza primero el cálculo del valor neto de matrícula. Este valor se obtiene al aplicar becas o descuentos, cuando corresponde. Luego se calcula la cuota inicial con base en el porcentaje mínimo definido por el tipo de programa. Después se determina el valor financiado, que corresponde a la matrícula neta menos la cuota inicial. Finalmente, según la modalidad seleccionada, el sistema calcula cuotas con o sin intereses.

Para la modalidad con intereses, el sistema genera una tabla de amortización donde cada fila muestra el comportamiento del saldo. Esta tabla permite al usuario identificar cuánto paga en cada cuota, qué parte corresponde a capital, qué parte corresponde a intereses y cuál es el saldo restante después de cada pago.

Ecuación 1. Valor neto de matrícula

Valor neto = Valor de matrícula - Beca o descuento

Ecuación 2. Valor financiado

Valor financiado = Valor neto - Cuota inicial

Ecuación 3. Cuota con interés bajo amortización francesa

$$A = P\frac{i(1+i)^n}{(1+i)^n - 1}$$

Donde A representa la cuota periódica, P representa el valor financiado, i representa la tasa de interés por periodo y n representa el número de cuotas.

4.9 Arquitectura del software

La arquitectura del sistema se diseñó con separación entre frontend, backend, base de datos e infraestructura. El frontend se encargó de la experiencia de usuario y la

visualización de resultados. El backend concentró la lógica de negocio, la validación de reglas, la autenticación, la generación de simulaciones y la comunicación con la base de datos. La base de datos almacenó usuarios, roles, parámetros financieros, simulaciones y registros necesarios para la trazabilidad.

La arquitectura también permitió documentar y probar los servicios mediante Swagger. Esto facilitó que el equipo pudiera revisar endpoints, validar entradas y salidas, y realizar pruebas técnicas sin depender siempre del frontend. Además, el uso de TypeScript en frontend y backend ayudó a reducir errores por tipos de datos y a mantener una estructura de código más clara.

La figura 6 representa la arquitectura general del sistema, en la cual se observa la separación entre frontend, backend, base de datos y servicios de autenticación.

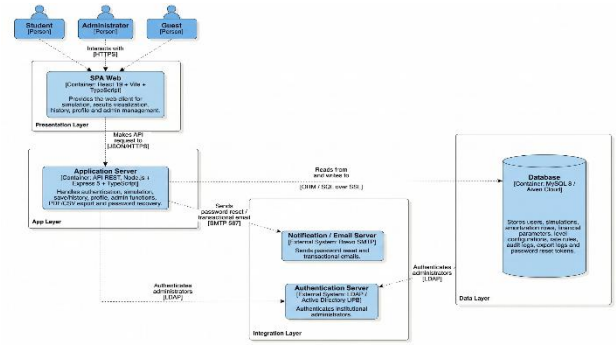


Figura 6. Arquitectura general del Simulador de Crédito Educativo.
Fuente: Diagrama de contenedores Nivel C4, autoría propia, 2026.

4.10 Modelo de datos

El modelo de datos se diseñó para soportar la operación del simulador. Entre las entidades principales se consideraron usuarios, roles, estudiantes, administradores, simulaciones, parámetros financieros, tasas, exportaciones y registros de auditoría. El modelo entidad-relación y el modelo relacional permitieron establecer cómo se conectaban las entidades y qué información debía persistirse.

La base de datos fue especialmente relevante para el módulo administrativo, ya que las reglas financieras no debían quedar quemadas en el código. Gracias a la parametrización, el administrador puede activar, desactivar o modificar condiciones según las decisiones del área financiera. Esta característica mejora la mantenibilidad del sistema porque reduce la necesidad de intervención técnica ante cambios de política.

La figura 7 presenta el modelo entidad-relación utilizado para estructurar los datos principales del sistema.

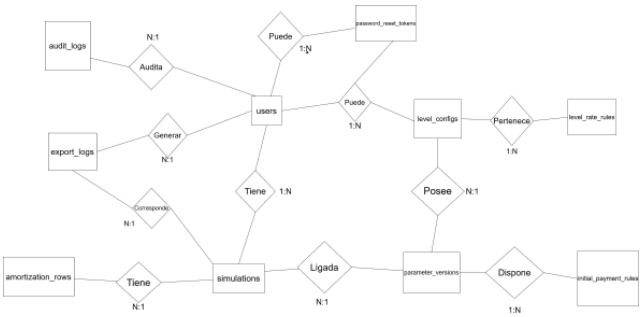


Figura 7. Modelo entidad-relación del sistema.
Fuente: Elaboración propia, 2026.

4.11 Pruebas del sistema

La validación del sistema incluyó pruebas unitarias, pruebas funcionales, pruebas de integración y pruebas de regresión básica. Las pruebas unitarias se enfocaron en componentes específicos, especialmente el motor de cálculo financiero. Las pruebas funcionales validaron que los flujos principales del sistema cumplieran lo esperado desde la perspectiva del usuario. Las pruebas de integración verificaron la comunicación entre frontend, backend, base de datos y servicios externos. Las pruebas de regresión básica permitieron confirmar que los cambios realizados no dañaran funcionalidades previamente validadas.

Tabla 7. Tipos de prueba aplicados

Tipo	Propósito	Validación
Unitaria	Validar funciones	Cuota inicial e intereses
Funcional	Verificar flujos	Simulación y guardado
Integración	Revisar módulos	Frontend + API + BD
Regresión	Verificar cambios	Repetición de flujos
API	Validar endpoints	Postman y Swagger
Despliegue	Verificar producción	URL, PM2 y Nginx

Fuente: Plan y reporte de pruebas del sistema, Autores, 2026.

4.12 Despliegue institucional

El despliegue final permitió que el sistema estuviera disponible mediante una URL institucional: https://simuladorfinanciaciondirecta.bucaramanga.upb.edu.co/

El proceso de despliegue incluyó configuración de servidor, variables de entorno, backend, frontend, base de datos, Nginx, PM2, certificados, servicios de red y coordinación con el área encargada de infraestructura. Antes del

despliegue final se realizó una etapa de preproducción que permitió ajustar variables, validar servicios y preparar el sistema para que el despliegue con CTIC fuera más rápido y controlado.

La diferencia principal entre la preproducción y el despliegue final fue que en preproducción se validaron componentes en un entorno controlado, mientras que en producción el sistema quedó alojado en infraestructura institucional y accesible desde redes externas. Esto implicó mayor cuidado en seguridad, configuración de dominio, disponibilidad, manejo de procesos y conexión con servicios institucionales.

La figura 8 muestra el despliegue final del sistema en la infraestructura institucional.

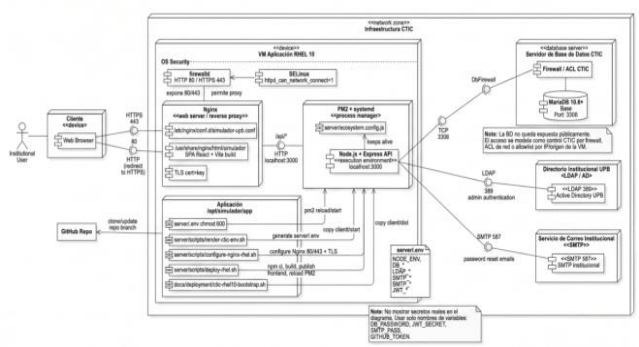


Figura 8. Diagrama de despliegue del sistema.
Fuente: Diagrama de despliegue, autoría propia, 2026.

4.13 Validación con el cliente

El proyecto fue validado mediante reuniones con el área financiera y evidencias documentadas en actas. Estas actas registraron retroalimentación, ajustes solicitados, compromisos, validaciones parciales y aprobación progresiva del sistema. La validación con el cliente fue fundamental porque varias reglas de negocio cambiaron durante el desarrollo, especialmente las relacionadas con porcentajes, modalidades, cuotas, becas, descuentos y parametrización.

La entrega final permitió presentar el producto funcionando, revisar la URL productiva, validar el acceso como estudiante y administrador, probar flujos de simulación, verificar cálculos financieros, revisar la tabla de amortización y confirmar el funcionamiento de los módulos principales. Esta validación permitió concluir que el sistema respondía a la necesidad planteada inicialmente.

La figura 9 evidencia una de las actas firmadas empleadas para la validación y seguimiento del proyecto.

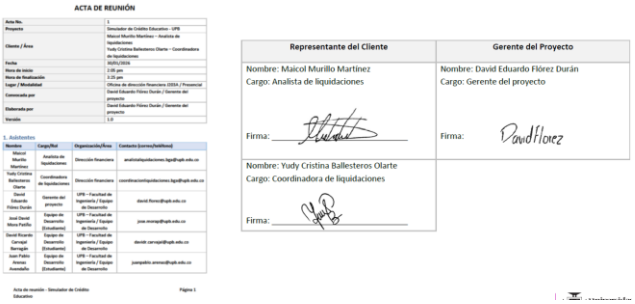


Figura 9. Acta de reunión firmada para validación y seguimiento del proyecto.
Fuente: Actas de reunión del proyecto, Autores, 2026.

5. Discusión de resultados

Los resultados obtenidos muestran que una plataforma web de simulación financiera puede mejorar la orientación de los aspirantes y estudiantes frente a sus posibilidades de financiación. Antes del desarrollo del sistema, muchas consultas dependían de la atención directa del área financiera. Con el simulador, el usuario puede ingresar datos básicos, comparar escenarios y obtener una tabla de amortización sin esperar una respuesta manual.

Desde el punto de vista funcional, el resultado más importante fue la integración del motor de cálculo con una interfaz comprensible. No era suficiente calcular correctamente; también era necesario presentar los resultados de forma clara. Por eso se incluyeron resúmenes financieros, tablas y elementos visuales que facilitan la interpretación. Esto se relaciona con el objetivo del proyecto, ya que la plataforma busca apoyar la toma de decisiones y no únicamente procesar datos.

Desde el punto de vista técnico, la decisión de separar frontend, backend y base de datos fue adecuada porque permitió organizar responsabilidades y facilitar el mantenimiento. El frontend pudo enfocarse en la experiencia de usuario, mientras el backend concentró reglas de negocio, validaciones, seguridad y persistencia. Además, el uso de una API REST documentada con Swagger hizo más sencilla la prueba de endpoints y la comunicación entre integrantes del equipo.

La parametrización financiera fue otro aporte importante. En sistemas administrativos, uno de los problemas frecuentes es que las reglas cambian con el tiempo. Si estas reglas quedan escritas directamente en el código, cada cambio requiere intervención técnica. En cambio, al permitir que el administrador gestione parámetros, tasas y vigencias, el sistema se vuelve más flexible y útil para el área financiera.

En cuanto a la metodología, el uso de prototipado evolutivo y artefactos de Scrum permitió avanzar por entregas y ajustar el producto con base en la retroalimentación del cliente. Esto fue positivo porque el proyecto tuvo cambios de alcance y ajustes de reglas de negocio durante su desarrollo. Las reuniones diarias, los tableros de GitHub y los Sprint Backlogs ayudaron a mantener trazabilidad sobre lo que se estaba construyendo.

Sin embargo, también se presentaron limitaciones. La primera fue la dependencia de configuraciones externas para el despliegue institucional, especialmente en temas de red, servidor, LDAP, dominio y servicios de infraestructura. La segunda fue el tiempo disponible para pruebas avanzadas. Aunque se realizaron pruebas unitarias, funcionales, de integración y regresión, no se ejecutaron pruebas profundas de carga, estrés o seguridad ofensiva. Esto no invalida el producto, pero sí deja una oportunidad de mejora para futuras versiones.

Otra limitación fue la variación de requerimientos durante el proyecto. Algunas reglas financieras se ajustaron después de tener funcionalidades en desarrollo, lo que obligó al equipo a rehacer partes del flujo, modificar validaciones y adaptar la base de datos. Este aprendizaje evidencia la importancia de delimitar mejor el alcance desde etapas tempranas y mantener una comunicación clara con el cliente.

En términos generales, el proyecto cumplió su propósito. La plataforma quedó funcional, desplegada y validada, con un cumplimiento aproximado del 98 % del alcance planificado. El sistema no reemplaza completamente la asesoría financiera institucional, pero sí automatiza la orientación inicial y reduce la carga de consultas repetitivas. Además, deja una base tecnológica que puede ampliarse con nuevas funciones, integraciones y reportes administrativos.

6. Conclusiones

El desarrollo del Simulador de Crédito Educativo permitió implementar una plataforma web funcional para apoyar la orientación financiera de aspirantes y estudiantes de la Universidad Pontificia Bolivariana Seccional Bucaramanga. La solución responde a una necesidad real del área financiera: disminuir la atención manual de consultas repetitivas y ofrecer al usuario una herramienta clara para proyectar sus pagos.

El motor de cálculo financiero fue el componente central del sistema, ya que permitió calcular matrícula neta, cuota inicial, valor financiado, cuotas, intereses y tabla de amortización. Su integración con la interfaz web facilitó que los resultados fueran comprensibles para usuarios no técnicos.

El uso de una arquitectura separada entre frontend, backend y base de datos favoreció la mantenibilidad del sistema. React, TypeScript, Node.js, Express, MySQL y Sequelize permitieron construir una solución modular, mientras que JWT, BCrypt, roles y LDAP fortalecieron el control de acceso.

La parametrización administrativa fue uno de los principales aportes del proyecto, porque permite modificar reglas financieras sin intervenir directamente el código fuente. Esto mejora la flexibilidad del sistema frente a cambios futuros en tasas, vigencias o políticas institucionales.

La metodología incremental con prototipado evolutivo y artefactos de Scrum permitió organizar el trabajo del equipo, medir avances y responder a cambios solicitados por el cliente. Los tres sprints permitieron pasar de una base técnica inicial a un producto desplegado y validado.

Las pruebas unitarias, funcionales, de integración y de regresión básica permitieron comprobar la precisión de los cálculos y la estabilidad general de los flujos principales. Aunque no se realizaron pruebas avanzadas de carga o estrés, el alcance de pruebas fue suficiente para validar el funcionamiento académico e institucional del producto final.

El despliegue en una URL institucional demostró que el sistema superó la etapa de prototipo local y alcanzó un ambiente real de operación. Esto representa un resultado importante para el proyecto, porque la plataforma quedó disponible para ser usada y evaluada más allá del entorno de desarrollo.

Finalmente, el proyecto evidencia que la tecnología puede transformar procesos administrativos universitarios cuando se diseña a partir de necesidades reales, validación continua con el cliente y una arquitectura pensada para evolucionar. Como trabajo futuro, se recomienda fortalecer métricas de uso, pruebas de rendimiento, reportes administrativos avanzados, analítica de simulaciones y nuevas integraciones con sistemas institucionales.

7. Referencias bibliográficas

American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). American Psychological Association.

Andrade, L. (2022). Backend básico: RESTful API. Medium.

Banco de la República. (s. f.). ¿Qué es la tasa de interés? Banco de la República de Colombia.

Congreso de Colombia. (2008). Ley 1266 de 2008. Por la cual se dictan disposiciones generales del habeas data y se regula el manejo de la información contenida en bases de datos personales.

Congreso de Colombia. (2012). Ley 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales.

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Fowler, M. (s. f.). Data Transfer Object. Martin Fowler.

GitHub Docs. (s. f.). GitHub Actions documentation.

Gitman, L. J., & Zutter, C. J. (2012). Principios de administración financiera. Pearson.

ICETEX. (s. f.). Crédito educativo. Instituto Colombiano de Crédito Educativo y Estudios Técnicos en el Exterior.

IEEE. (1998). IEEE recommended practice for software requirements specifications. IEEE Std 830-1998.

JWT.io. (s. f.). Introduction to JSON Web Tokens.

Nx. (s. f.). Monorepos and smart build systems.

OWASP Foundation. (s. f.). OWASP Cheat Sheet Series.

Pressman, R. S., & Maxim, B. R. (2015). Ingeniería del software: un enfoque práctico. McGraw-Hill.

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide.

Sequelize. (s. f.). Sequelize ORM documentation.

The Twelve-Factor App. (s. f.). Config.

The Twelve-Factor App. (s. f.). Build, release, run.

Universidad Pontificia Bolivariana. (2026). Actas de reunión del Proyecto Simulador de Crédito Educativo. Documento interno.

Universidad Pontificia Bolivariana. (2026). Documento de especificación de requisitos de software del Simulador de Crédito Educativo. Documento interno.

Universidad Pontificia Bolivariana. (2026). Product Backlog y Sprint Backlogs del Proyecto Simulador de Crédito Educativo. Documento interno.

Universidad Pontificia Bolivariana. (2026). Plan y reporte de pruebas del Simulador de Crédito Educativo. Documento interno.

Universidad Pontificia Bolivariana. (2026). Presentación final del Simulador de Crédito Educativo. Documento interno.